

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 9

Integrasi AI dengan Gemini

Merancang prompt, memanggil Gemini dengan keluaran JSON terstruktur, dan menampilkan hasilnya sebagai saran yang bisa dikoreksi

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 9. Tahun 2026

Komponen	Keterangan
Pertemuan	9
Capaian	CPMK0503, menerapkan teknologi sistem cerdas dalam pengembangan perangkat lunak
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 9

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: judul seadanya, admin yang menebak
3	Langkah 1: Empat bagian prompt
4	Langkah 2: Prompt seadanya dan prompt LaporKampus
5	Langkah 3: Prompt disusun oleh kode
6	Langkah 4: Isi permintaan ke generateContent
7	Langkah 5: Skema dan jawaban sebagai data class
8	Langkah 6: Kunci API tidak masuk repository
9	Langkah 7: AsistenGemini memanggil generateContent
10	Langkah 8: Satu interface, dua implementasi
11	Langkah 9: AI yang menyarankan, bukan memutuskan
12	Langkah 10: Sepuluh kasus uji sebelum percaya
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 9. Tab Asisten di LaporKampus sudah kosong sejak Modul 5. Di modul ini kita mengisinya dengan fitur AI yang membantu pelapor: dari deskripsi bebas, AI menyarankan kategori, gedung, dan tingkat urgensi laporan. Keputusan akhirnya tetap di tangan pelapor.

Modul ini terdiri dari tiga bagian. Langkah 1 sampai 3 membahas cara menyusun **prompt** yang bisa diandalkan. Langkah 4 sampai 8 membahas cara **memanggil Gemini** dari Kotlin Multiplatform

dengan Ktor, meminta keluaran JSON dengan skema, dan menjaga kunci API tetap aman. Langkah 9 dan 10 membahas cara **menampilkan hasil AI sebagai saran** dan cara mengukur seberapa sering saran itu benar.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menyusun prompt dengan peran, konteks, format, dan contoh	Langkah 1 sampai 3
2	Memanggil Gemini dari KMP dengan keluaran JSON sesuai skema, dengan kunci API yang tidak masuk repository	Langkah 4 sampai 8
3	Menampilkan hasil AI sebagai saran yang bisa dikoreksi, menangani error, dan mengukur akurasi	Langkah 9 dan 10

Capaian ini mendukung **CPMK0503**: menerapkan teknologi sistem cerdas dalam pengembangan perangkat lunak. Ini satu-satunya pertemuan yang langsung menyorot CPMK0503, dan bahannya dipakai lagi di **Laporan hasil proyek** yang rubriknya ada di akhir modul ini.

Yang perlu disiapkan

Proyek LaporKampus dari Modul 8 dan akun Google untuk membuka **Google AI Studio** di aistudio.google.com. Kunci API dibuat di halaman aistudio.google.com/apikey. Latihan di repo tetap bisa dikerjakan tanpa kunci, karena memakai jawaban tiruan.

PENTING Mengingat modul sebelumnya

Kita memakai `HttpClient`, `DTO`, dan `body()` dari Modul 6, interface dengan versi palsu dari Modul 2, 6, dan 8, sealed interface untuk keadaan layar dari Modul 6, serta `ViewModel` dan `StateFlow` dari Modul 4. Pola `CancellationException` yang dilempar ulang dari Modul 2 juga muncul lagi di Langkah 7.

PEMBUKA

Masalah pembuka: judul seadanya, admin yang menebak

Laporan seperti ini masuk setiap hari:

Kolom	Isi dari pelapor
Judul	rusak
Deskripsi	AC di GKU 1 R201 netes terus ke meja dosen, lantainya jadi licin
Kategori	Belum dipilih

Judulnya seadanya dan kategorinya kosong, sehingga admin harus menebak. Padahal deskripsinya sudah cukup jelas.

TEBAK DULU **Coba pikirkan**

Dari deskripsi di atas, apa kategori, gedung, dan tingkat urgensinya? Lalu, bisakah AI membantu menebaknya tanpa mengambil alih keputusan pengguna? Kalau AI salah menebak, siapa yang menanggung akibatnya?

JAWABAN **Arah jawabannya**

Kategori AC, gedung GKU 1, dan urgensi kemungkinan Tinggi karena lantai licin bisa membuat orang terpeleset. Kalian bisa menebaknya dalam dua detik, dan itu pertanda tugas ini cocok untuk AI. Tetapi AI bisa salah, dan yang menanggung akibatnya adalah pelapor dan petugas. Karena itu, di modul ini hasil AI selalu tampil sebagai **saran** yang bisa diubah atau diabaikan, bukan keputusan.

LANGKAH 1

Empat bagian prompt

Prompt adalah teks perintah yang kita kirim ke model AI. Prompt yang bisa diandalkan punya empat bagian:

Bagian	Isinya	Di Laporkampus
Peran	Siapa AI di sini	Kamu membantu admin fasilitas kampus ITERA

Bagian	Isinya	Di Laporkampus
Konteks	Yang perlu ia tahu	Daftar gedung, daftar kategori, dan arti urgensi Tinggi
Format	Bentuk jawaban	Hanya JSON dengan field kategori, gedung, dan urgensi
Contoh	Satu kasus beserta jawaban yang diharapkan	Lampu R305 GKU 2 kedip berarti Listrik, GKU 2, Rendah

Deskripsi dari pelapor ditaruh paling akhir, setelah semua aturan. Yang menyusun prompt adalah aplikasi, bukan pelapor. Di hands-on P10, bagian peran ini disebut **instruksi sistem**.

ANALOGI **Asisten lab baru**

Bayangkan kalian memberi tugas ke asisten lab yang baru masuk. Kalau hanya dibilang tolong cek laporan ini, ia akan bertanya balik: laporan apa, dicek bagaimana, hasilnya ditulis di mana? Kalau diberi tahu perannya, daftar gedungnya, bentuk catatannya, dan satu contoh catatan yang benar, hasil kerjanya langsung bisa dipakai.

LANGKAH 2

Prompt seadanya dan prompt Laporkampus

TEBAK DULU

Prompt kita hanya berbunyi: Tentukan kategori laporan ini: rusak. Apa jawaban AI? Kalau dikirim tiga kali, apakah jawabannya sama?

JAWABAN

AI biasanya menjawab dengan paragraf, menyebut beberapa kemungkinan, atau bertanya balik meminta detail. Tiga kali kirim bisa memberi tiga susunan kalimat yang berbeda. Bagi manusia itu wajar, tetapi bagi kode itu masalah: tidak ada field yang bisa diambil. Coba langsung di AI Studio supaya kalian melihatnya sendiri.

Bagian	Prompt seadanya	Prompt Laporkampus
Peran	Tidak ada	Admin fasilitas kampus ITERA
Konteks	Tidak ada	Daftar gedung, kategori, dan arti urgensi
Format	Bebas	Hanya JSON sesuai skema
Contoh	Tidak ada	Satu deskripsi dengan jawabannya

Bagian	Prompt seadanya	Prompt LaporanKampus
Hasilnya	Paragraf yang berbeda setiap kali	Tiga field yang bisa dibaca kode

Baris terakhir adalah intinya. Kita tidak sedang membuat AI lebih pintar. Kita membuat jawabannya bisa dipakai program. Konteks berupa daftar gedung juga mencegah AI mengarang nama gedung yang tidak ada di ITERA.

CONTOH Contoh sehari-hari

Bandingkan meminta teman menulis surat izin sakit dengan bebas, dan meminta teman mengisi formulir izin yang kolomnya sudah tercetak: nama, tanggal, alasan. Surat bebas isinya bisa macam-macam dan sulit direkap. Formulir bisa langsung dimasukkan ke tabel rekap. Format JSON adalah formulirnya.

LANGKAH 3

Prompt disusun oleh kode

Daftar pilihan yang dipakai di form, di prompt, dan nanti di skema kita tulis satu kali saja.

commonMain/ai/Pilihan.kt

```
val KATEGORI = listOf("AC", "Listrik", "Elektronik kelas", "Bangunan", "Air",
    "Perabot")
val GEDUNG = listOf("GKU 1", "GKU 2", "Labtek 1", "Labtek 3", "Perpustakaan", "Tidak
    disebut")
val URGENSI = listOf("Rendah", "Sedang", "Tinggi")
```

Daftar di atas hanya contoh. Isi dengan pilihan yang sama persis dengan form laporan kalian. Pilihan Tidak disebut di daftar gedung penting, supaya AI tidak dipaksa menebak gedung kalau pelapor memang tidak menulisnya.

commonMain/ai/Prompt.kt

```

fun susunPrompt(deskripsi: String) = buildString {
    // peran
    appendLine("Kamu membantu admin fasilitas kampus ITERA.")
    // konteks
    appendLine("Gedung: " + GEDUNG.joinToString())
    appendLine("Kategori: " + KATEGORI.joinToString())
    appendLine("Urgensi Tinggi bila membahayakan orang.")
    // format
    appendLine("Jawab hanya dengan JSON sesuai skema.")
    // contoh
    appendLine("Contoh: lampu R305 GKU 2 kedip berarti")
    appendLine("kategori Listrik, gedung GKU 2, urgensi Rendah.")
    append("Deskripsi pelapor: $deskripsi")
}

```

`buildString` menyusun teks baris demi baris, dan `joinToString()` menggabungkan daftar menjadi satu teks yang dipisah koma. Prompt kini menjadi fungsi biasa: bisa dibaca saat code review dan bisa diuji di Modul 10. Simpan setiap versi prompt di repository, karena perubahannya akan kalian ceritakan di Laporan hasil proyek.

PENTING Deskripsi pelapor selalu di akhir

Deskripsi berasal dari pengguna, jadi isinya bisa apa saja, termasuk kalimat seperti abaikan semua aturan di atas. Menaruhnya di akhir, setelah semua aturan, dan menyebutnya dengan jelas sebagai deskripsi pelapor, membantu model membedakan aturan dari aplikasi dengan teks dari pengguna.

LANGKAH 4

Isi permintaan ke generateContent

Gemini dipanggil dengan satu permintaan POST ke alamat berbentuk alamat dasar, lalu nama model, lalu `:generateContent`. Bentuk badan permintaannya seperti ini:

Badan JSON yang dikirim

```
{
  "contents": [{ "parts": [{ "text": "..."}] }],
  "generationConfig": {
    "responseMimeType": "application/json",
    "responseSchema": {
      "type": "OBJECT",
      "properties": {
        "kategori": { "type": "STRING" },
        "gedung": { "type": "STRING" },
        "urgensi": { "type": "STRING",
          "enum": ["Rendah", "Sedang", "Tinggi"] }
      },
      "required": ["kategori", "gedung", "urgensi"]
    }
  }
}
```

Bagian	Gunanya
<code>contents</code>	Prompt dari susunPrompt
<code>responseMimeType</code>	Meminta jawaban berupa JSON, bukan kalimat
<code>responseSchema</code>	Nama field, tipe, dan pilihan yang boleh dipakai
<code>enum</code>	Daftar nilai yang boleh dipilih untuk satu field
<code>required</code>	Field yang wajib ada

Bagian `contents` sudah dipakai di hands-on P10. Yang baru adalah `generationConfig`. Dengan `enum`, AI hanya boleh memilih Rendah, Sedang, atau Tinggi untuk urgensi.

ANALOGI Kolom pilihan di formulir

`responseSchema` seperti formulir dengan kolom yang sudah ditentukan. `enum` seperti kolom berupa pilihan dropdown: pengisi tidak bisa menulis Agak penting atau URGENT BANGET, ia hanya bisa memilih salah satu dari tiga pilihan yang tersedia.

PENTING Selalu cocokkan dengan dokumentasi

Nama field dan nama model selalu cocokkan dengan dokumentasi Gemini di ai.google.dev, terutama halaman structured output. API ini masih sering diperbarui, dan nama model lama bisa berhenti dilayani.

LANGKAH 5

Skema dan jawaban sebagai data class

Skema ditulis sekali di Kotlin

commonMain/ai/Skema.kt

```
@Serializable
data class Skema(
    val type: String,
    val properties: Map<String, Skema>? = null,
    val enum: List<String>? = null,
    val required: List<String>? = null
)

val SKEMA_SARAN = Skema("OBJECT",
    properties = mapOf(
        "kategori" to Skema("STRING", enum = KATEGORI),
        "gedung" to Skema("STRING", enum = GEDUNG),
        "urgensi" to Skema("STRING", enum = URGENSI),
        "alasan" to Skema("STRING")
    ),
    required = listOf("kategori", "gedung", "urgensi")
)
```

Skema berbentuk data class, sehingga kotlinx.serialization yang mengubahnya menjadi JSON seperti di Langkah 4. Field yang bernilai null tidak ikut dikirim, karena sama dengan nilai bawaannya. Daftar KATEGORI yang sama kini dipakai di tiga tempat: form, prompt, dan skema. Kalau kategori baru ditambah, ketiganya ikut berubah.

Gedung juga diberi enum supaya AI tidak menulis Gedung Kuliah Umum 1 di satu jawaban dan GKU1 di jawaban lain. Field alasan tidak wajib, dan isinya ditampilkan kecil di kartu saran.

Jawaban menjadi data class

commonMain/ai/SaranLaporan.kt

```
@Serializable
data class SaranLaporan(
    val kategori: String,
    val gedung: String,
    val urgensi: String,
    val alasan: String = ""
)
```

JSON dari Gemini tidak langsung berada di badan jawaban. Ia terbungkus di dalam teks: candidates, lalu content, lalu parts, lalu text. Jadi kita membaca dua kali.

commonMain/ai/GeminiDto.kt

```

@Serializable data class Part(val text: String)
@Serializable data class Content(val parts: List<Part>)
@Serializable data class Candidate(val content: Content? = null)
@Serializable data class GeminiResponse(val candidates: List<Candidate>? = null)

@Serializable data class GenerationConfig(
    val responseMimeType: String,
    val responseSchema: Skema
)
@Serializable data class GeminiRequest(
    val contents: List<Content>,
    val generationConfig: GenerationConfig
)

fun permintaan(prompt: String) = GeminiRequest(
    contents = listOf(Content(listOf(Part(prompt)))),
    generationConfig = GenerationConfig("application/json", SKEMA_SARAN)
)

```

Bentuk Part, Content, Candidate, dan GeminiResponse sama dengan hands-on P10. Candidate dan candidates dibuat nullable karena jawaban bisa kosong, misalnya saat prompt terkena filter keamanan. Pembacaan pertama mengubah badan HTTP menjadi GeminiResponse. Pembacaan kedua mengubah teks di dalamnya menjadi SaranLaporan.

ANALOGI Amplop di dalam amplop

Jawaban Gemini seperti paket kiriman: kardus luar berisi label pengiriman, dan di dalamnya ada amplop berisi surat yang kita tunggu. Kita membuka kardusnya dulu dengan GeminiResponse, baru membuka amplopnya dengan SaranLaporan.

LANGKAH 6

Kunci API tidak masuk repository

Kunci API adalah tanda pengenal yang membuat Gemini mau melayani permintaan kita, sekaligus menghitung kuota pemakaian kita. Siapa pun yang memegang kunci itu bisa memakai kuota kalian.

```
local.properties, sudah ada di .gitignore
```

```
GEMINI_API_KEY=isi-kunci-dari-ai-studio
```

```
composeApp/build.gradle.kts
```

```
import java.util.Properties

val lokal = Properties().apply {
    val f = rootProject.file("local.properties")
    if (f.exists()) f.inputStream().use { load(it) }
}

android {
    buildFeatures { buildConfig = true }
    defaultConfig {
        buildConfigField("String", "GEMINI_API_KEY",
            "\"\${lokal.getProperty("GEMINI_API_KEY", "")}\"")
    }
}
```

Baris `import java.util.Properties` ditaruh paling atas berkas Gradle. Saat build, kunci dari `local.properties` disalin ke kelas `BuildConfig`, sehingga kode Android bisa membacanya tanpa kunci itu tertulis di kode sumber.

Di mana	Cara membaca kunci
Android	<code>BuildConfig.GEMINI_API_KEY</code>
Desktop	<code>System.getenv("GEMINI_API_KEY")</code>
Rilis sungguhan	Server perantara milik kelompok, kunci tidak ada di aplikasi

Kunci dikirim ke `AsistenGemini` lewat constructor, sama seperti implementasi platform di Modul 8. Misalnya di `MainActivity`: `AsistenGemini(client, BuildConfig.GEMINI_API_KEY)`.

ANALOGI Kunci kamar kos

Menulis kunci API di kode yang di-push ke GitHub seperti menempel kunci kamar kos di papan pengumuman. Menaruhnya di `BuildConfig` seperti menyimpan kunci di bawah keset: tidak terlihat orang lewat, tetapi orang yang sengaja mencari tetap bisa menemukannya, karena kunci di `BuildConfig` masih bisa dibongkar dari berkas APK. Cara paling aman adalah kunci yang dipegang penjaga kos, yaitu server perantara. Untuk tugas kuliah ini, `BuildConfig` sudah cukup, asal kuncinya tidak pernah ada di riwayat commit.

PENTING Kalau kunci terlanjur di-push

Hapus kunci itu di AI Studio dan buat kunci baru. Menghapus commit saja tidak cukup, karena riwayat repository bisa sudah disalin orang lain. Kalau ragu, periksa dengan `git log -p` dan cari kata `GEMINI_API_KEY`.

LANGKAH 7

Asisten Gemini memanggil generateContent

commonMain/ai/AsistenGemini.kt

```

const val GEMINI_BASE_URL = "https://generativelanguage.googleapis.com/v1beta/models"
const val GEMINI_MODEL = "..." // isi sesuai daftar model di dokumentasi Gemini
const val URL_GEMINI = "$GEMINI_BASE_URL/$GEMINI_MODEL:generateContent"

class AsistenGemini(
    private val client: HttpClient,
    private val kunci: String
) : AsistenLaporan {
    private val json = Json { ignoreUnknownKeys = true }

    override suspend fun sarankan(isi: String): Result<SaranLaporan> = try {
        val jawab: GeminiResponse = client.post(URL_GEMINI) {
            parameter("key", kunci)
            contentType(ContentType.Application.Json)
            setBody(permintaan(susunPrompt(isi)))
        }.body()
        val teks = jawab.candidates?.firstOrNull()
            ?.content?.parts?.firstOrNull()?.text
            ?: error("Gemini tidak memberi jawaban")
        Result.success(json.decodeFromString<SaranLaporan>(teks))
    } catch (e: CancellationException) {
        throw e
    } catch (e: Exception) {
        Result.failure(e)
    }
}

```

Bagian	Artinya
URL_GEMINI	Alamat dasar, nama model, lalu :generateContent
parameter("key", kunci)	Kunci API dikirim sebagai parameter alamat
permintaan(...)	Membungkus prompt dan SKEMA_SARAN
Rantai ?.	Membuka candidates, content, parts, lalu text. Kalau ada yang kosong, hasilnya null
error(...)	Melempar exception dengan pesan, ditangkap catch di bawahnya
Result	Wadah yang berisi hasil berhasil atau error, tanpa melempar exception ke pemanggil

HttpClient yang dipakai sama dengan Modul 6, dengan `expectSuccess = true` dan `HttpTimeout`, sehingga status 4xx, 5xx, dan timeout ikut menjadi `Result.failure`. Beri timeout lebih longgar dari API biasa, misalnya 15 detik, karena model butuh waktu untuk menyusun jawaban.

PENTING runCatching dan pembatalan

Di slide dan di banyak tutorial, isi fungsi ini ditulis dengan `runCatching { ... }` yang lebih ringkas. Hati-hati: `runCatching` ikut menangkap `CancellationException`. Di dalam coroutine, itu membuat pembatalan tertelan, persis masalah yang kita bahas di Langkah 10 Modul 2. Karena itu di modul ini kita memakai `try` dan `catch` yang melempar ulang `CancellationException`.

TEBAK DULU

`responseMimeType` dan skema kita hapus dari permintaan, tetapi prompt tetap meminta JSON. Lalu kita panggil `decodeFromString` pada jawabannya. Apakah selalu berhasil?

JAWABAN

Kadang berhasil, kadang gagal dengan `SerializationException`. Tanpa `responseMimeType`, jawaban bisa dibungkus tanda kode markdown, diawali kalimat pengantar seperti Berikut hasilnya, atau memakai nama field lain seperti `category`. Materi lama bahkan memakai `removePrefix` untuk membuang pembungkus itu. Dengan `responseMimeType` dan skema, trik seperti itu tidak perlu lagi.

LANGKAH 8

Satu interface, dua implementasi

commonMain/ai/AsistenLaporan.kt

```
interface AsistenLaporan {
    suspend fun sarankan(isi: String): Result<SaranLaporan>
}

class AsistenPalsu : AsistenLaporan {
    override suspend fun sarankan(isi: String): Result<SaranLaporan> {
        delay(1000)
        return Result.success(SaranLaporan("AC", "GKU 1", "Sedang", "data palsu"))
    }
}
```

LaporKampus	Hands-on P10
AsistenLaporan	AiRepository
AsistenPalsu	MockAiRepository
AsistenGemini	RealAiRepository

Polanya sama dengan Sumber dan SumberLaporanPalsu sejak Modul 2. Layar Asisten bisa dikembangkan dengan versi palsu, tanpa kunci dan tanpa memakai kuota. Satu anggota kelompok bisa membuat layar Asisten dengan AsistenPalsu, sementara anggota lain menyelesaikan AsistenGemini. Penggantiannya cukup di satu baris tempat objek dibuat. Di Modul 10, versi palsu ini juga dipakai di unit test.

ANALOGI Gladi bersih

AsistenPalsu seperti gladi bersih acara wisuda dengan pembicara pengganti. Semua urutan acara, mikrofon, dan tata panggung bisa diuji tanpa menunggu pembicara aslinya datang. Saat hari H, pembicara asli tinggal naik panggung.

LANGKAH 9

AI yang menyarankan, bukan memutuskan

Empat keadaan layar Asisten

commonMain/ui/AsistenUiState.kt

```
sealed interface AsistenUiState {
    data object Kosong : AsistenUiState
    data object Memuat : AsistenUiState
    data class Saran(val isi: SaranLaporan) : AsistenUiState
    data class Gagal(val pesan: String) : AsistenUiState
}
```

Keadaan	Yang tampil
Kosong	Kolom deskripsi dan tombol Minta saran
Memuat	Indikator memuat, tombol nonaktif
Saran	Kartu Saran AI dengan tombol Pakai dan Abaikan
Gagal	Pesan singkat, form tetap bisa diisi

Ini pola sealed interface dari Modul 6, ditambah keadaan Kosong sebelum pelapor meminta apa pun. Di keadaan Gagal, pelapor tetap harus bisa mengirim laporan. AI adalah bantuan, bukan syarat.

AsistenViewModel

commonMain/ui/AsistenViewModel.kt

```

class AsistenViewModel(private val asisten: AsistenLaporan) : ViewModel() {
    private val _ui = MutableStateFlow<AsistenUiState>(AsistenUiState.Kosong)
    val ui = _ui.asStateFlow()

    fun minta(deskripsi: String) {
        if (_ui.value == AsistenUiState.Memuat) return
        viewModelScope.launch {
            _ui.value = AsistenUiState.Memuat
            _ui.value = asisten.sarankan(deskripsi).fold(
                onSuccess = { AsistenUiState.Saran(it) },
                onFailure = { AsistenUiState.Gagal("Saran AI belum bisa dibuat. Isi
form secara manual.") }
            )
        }
    }

    fun abaikan() { _ui.value = AsistenUiState.Kosong }
}

```

ViewModel hanya tahu AsistenLaporan. Ia tidak tahu ada Gemini, Ktor, atau kunci API. `fold` memilih satu dari dua cabang: `onSuccess` kalau Result berisi hasil, `onFailure` kalau berisi error. Pesan untuk pengguna dibuat pendek, sedangkan detail error sebaiknya dicatat ke log untuk developer. Pemeriksaan di awal fungsi `minta` mencegah satu laporan memakai kuota dua kali saat tombol ditekan berulang.

Kartu saran yang bisa dikoreksi

commonMain/ui/KartuSaran.kt

```

@Composable
fun KartuSaran(saran: SaranLaporan, onPakai: () → Unit, onAbaikan: () → Unit) {
    Card {
        Column(Modifier.padding(16.dp), verticalArrangement =
Arrangement.spacedBy(4.dp)) {
            Text("Saran AI", fontWeight = FontWeight.Bold)
            Text("Kategori: ${saran.kategori}")
            Text("Gedung: ${saran.gedung}")
            Text("Urgensi: ${saran.urgensi}")
            if (saran.alasan.isNotBlank()) Text(saran.alasan, fontSize = 12.sp, color =
Color.Gray)
            Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                TextButton(onClick = onAbaikan) { Text("Abaikan") }
                Button(onClick = onPakai) { Text("Pakai") }
            }
        }
    }
}

```

Prinsip	Artinya
Diberi label	Pelapor tahu isian ini dari AI, bukan dari admin
Bisa diubah	Pakai menyalin saran ke form, dan setiap kolom tetap bisa diubah

Prinsip	Artinya
Tidak tersimpan otomatis	Laporan baru terkirim saat pelapor sendiri menekan Kirim

Tombol Pakai hanya memanggil fungsi ubah di `FormLaporanViewModel`, misalnya `ubahKategori`, `ubahLokasi`, dan `ubahUrgensi`. Kalau `UiState` form kalian belum punya kolom kategori dan urgensi, tambahkan dulu dengan pola yang sama seperti judul dan lokasi di Modul 4. Tombol Abaikan menutup kartu tanpa mengubah apa pun.

ANALOGI **Saran rute di aplikasi peta**

Aplikasi peta menyarankan rute tercepat, tetapi kalian tetap boleh memilih jalan lain karena tahu ada jalan ditutup. Aplikasi tidak pernah menyetir sendiri. Kartu Saran AI bekerja dengan cara yang sama: AI menyarankan, pelapor yang memutuskan.

Saat AI gagal, dan data yang tidak dikirim

Keadaan	Yang dilakukan aplikasi
Timeout atau internet kampus mati	Pesan singkat dan tombol coba lagi, form tetap bisa diisi
Kuota habis, HTTP 429	Minta pelapor mencoba lagi nanti, jangan mengulang otomatis terus
Kunci salah atau ditolak	Pesan umum ke pengguna, detail ke log
Jawaban kosong atau JSON rusak	Anggap gagal, jangan mengisi form dengan tebakan

Kuota gratis Gemini terbatas per menit dan per hari, jadi saat demo kuotanya bisa saja habis. Siapkan `AsistenPalsu` sebagai cadangan demo.

PENTING **Jangan dikirim ke AI**

NIM, nomor HP, nama orang, dan foto wajah. Kirim deskripsi kerusakan saja, dan beri tahu pengguna bahwa teksnya diproses layanan AI. Deskripsi seperti AC di ruang Pak Budi rusak tidak perlu menyebut nama Pak Budi. Kalau aplikasi kelompok berpotensi mengirim data pribadi, tulis batasannya di Laporan hasil proyek.

LANGKAH 10

Sepuluh kasus uji sebelum percaya

Bagaimana kita tahu AI cukup sering benar? Bukan dengan mencoba sekali lalu puas. Kita siapkan **sepuluh kasus uji**, tulis jawaban yang benar lebih dulu, baru bandingkan dengan jawaban AI.

No	Deskripsi	Kategori benar	Urgensi benar
1	AC di GKU 1 R201 netes ke meja, lantai licin	AC	Tinggi
2	Lampu koridor Labtek 3 berkedip	Listrik	Rendah
3	Stopkontak R305 gosong	Listrik	Tinggi
4	Air hujan masuk lewat plafon	Bangunan	Sedang

Kolom jawaban benar diisi manusia **sebelum** melihat jawaban AI. Lalu jalankan kesepuluh kasus, hitung yang cocok, dan ulangi setiap kali prompt diubah. Pilih kasus yang beragam: yang mudah, yang ambigu seperti nomor 4, dan yang tidak menyebut gedung. Akurasi tanpa kasus yang ambigu memang terlihat bagus, tetapi menipu.

ANALOGI Kunci jawaban dibuat sebelum ujian

Dosen menyusun kunci jawaban sebelum memeriksa lembar ujian, bukan sesudahnya. Kalau kunci dibuat setelah melihat jawaban mahasiswa, tanpa sadar kunci itu bisa ikut menyesuaikan diri. Begitu pula kasus uji AI.

Menghitung akurasi

Jalankan kode ini di Kotlin Playground.

```
Main.kt
```

```

data class Kasus(val deskripsi: String, val benar: String, val ai: String)

val kasus = listOf(
    Kasus("AC netes ke meja, lantai licin", "AC", "AC"),
    Kasus("Lampu koridor berkedip", "Listrik", "Listrik"),
    Kasus("Stopkontak gosong", "Listrik", "Listrik"),
    Kasus("Air hujan masuk lewat plafon", "Bangunan", "Air"),
    Kasus("Proyektor tidak menyala", "Elektronik kelas", "Listrik"),
    Kasus("Kursi patah di R201", "Perabot", "Perabot"),
    Kasus("Keran wastafel bocor", "Air", "Air"),
    Kasus("Pintu R305 macet", "Bangunan", "Bangunan"),
    Kasus("AC tidak dingin", "AC", "AC"),
    Kasus("Meja dosen goyang", "Perabot", "Perabot")
)

fun main() {
    val cocok = kasus.count { it.benar == it.ai }
    println("Akurasi kategori: $cocok dari ${kasus.size} = ${cocok * 100 /
kasus.size}%")
    kasus.filter { it.benar != it.ai }.forEach {
        println("Salah: ${it.deskripsi}. Benar ${it.benar}, AI ${it.ai}")
    }
}

```

TEBAK DULU

Berapa akurasinya, dan kasus mana yang dicetak sebagai salah? Lalu, ubah jawaban AI pada kasus proyektor menjadi benar. Berapa akurasinya sekarang?

JAWABAN**Keluaran**

Akurasi kategori: 8 dari 10 = 80%

Salah: Air hujan masuk lewat plafon. Benar Bangunan, AI Air

Salah: Proyektor tidak menyala. Benar Elektronik kelas, AI Listrik

Setelah kasus proyektor diperbaiki, akurasinya menjadi 9 dari 10, yaitu 90%.

Dua kasus yang salah lebih berguna daripada delapan yang benar. Proyektor memang bisa dianggap urusan listrik, jadi mungkin daftar kategori kita yang kurang jelas, bukan AI-nya.

Bocor dari plafon menunjukkan prompt perlu aturan tambahan, misalnya air yang masuk dari atap termasuk Bangunan. Perbaiki prompt, jalankan ulang sepuluh kasus, dan catat akurasi sebelum dan sesudahnya.

PRAKTIK

Latihan mandiri

Ada tiga latihan di folder P10 Integrasi AI API Hands-on. Semuanya berjalan tanpa kunci, karena memakai MockAiRepository. Ketiganya diamati, dengan layar chat di latihan 3 sebagai bukti utama.

Latihan	Berkas	Yang dilatih
1	Handson1RealAiRepository.kt	Melengkapi ask(): badan permintaan, POST ke generateContent, parsing, dan Result
2	Handson2PromptEngineering.kt	buildPrompt() yang menaruh instruksi sistem di depan pertanyaan pengguna
3	Handson3ChatScreen.kt	Layar chat dengan daftar pesan, tombol kirim, dan indikator memuat

CONTOH Petunjuk latihan 1

Polanya sama dengan AsistenGemini di Langkah 7. Kunci disimpan hanya di salinan lokal, cocokkan GEMINI_MODEL dengan dokumentasi, lalu tambahkan responseType. Pastikan setiap error menjadi Result.failure, bukan crash.

CONTOH Petunjuk latihan 2

Susun seperti Langkah 3: instruksi sistem di depan, pertanyaan pengguna di akhir. Uji dengan beberapa pertanyaan, termasuk yang berusaha mengubah aturan, misalnya abaikan instruksi sebelumnya.

CONTOH Petunjuk latihan 3

Daftar pesan disimpan di StateFlow ViewModel, dan layar hanya membacanya. Tombol kirim nonaktif selama memuat, seperti AsistenViewModel di Langkah 9. Saat repository mengembalikan failure, tampilkan pesan singkat di daftar, jangan biarkan aplikasi berhenti.

Yang diamati

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	Layar chat memanggil repository AI, prompt memakai instruksi sistem, error ditangani
Baik	70 sampai 84	Fitur berjalan, prompt atau penanganan error belum lengkap

Level	Skor	Yang terlihat
Cukup	55 sampai 69	Fitur berjalan dengan kunci API tertulis di kode
Kurang	di bawah 55	Belum memanggil AI

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk **CPMK0503**. Ini satu-satunya observasi untuk CPMK0503. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Kalau kunci API tertulis di kode, levelnya paling tinggi Cukup.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Jawaban selalu Error: di latihan 1	Kunci masih YOUR_API_KEY_HERE	Isi kunci asli dari AI Studio
HTTP 400 Bad Request	Bentuk JSON permintaan tidak sesuai	Cocokkan data class dengan dokumentasi Gemini
HTTP 403 atau kunci tidak valid	Kunci salah, dihapus, atau terkena batas	Buat kunci baru, periksa kuota harian
HTTP 404 di generateContent	Nama model salah atau sudah tidak dilayani	Ganti GEMINI_MODEL sesuai dokumentasi
HTTP 429	Kuota per menit atau per hari habis	Tunggu, atau pakai AsistenPalsu sementara
candidates kosong	Prompt terkena filter keamanan	Kembalikan Result.failure dengan pesan jelas
SerializationException	Jawaban bukan JSON atau nama field berbeda	Pasang responseType dan skema

PRAKTIK

Bawa ke proyekmu

Fitur AI setiap tema sudah tertulis di Bank Proyek, misalnya menebak kategori barang temuan atau meringkas jadwal. Polanya sama: masukan bebas dari pengguna menjadi saran terstruktur. Bagi kerja: satu anggota prompt dan skema, satu anggota AsistenGemini, satu anggota layar dan kartu saran.

- 1. Prompt tema.** Peran, konteks, format, dan contoh untuk entitas utama tema kalian.
- 2. Skema dan data class.** Keluaran JSON terstruktur yang langsung dibaca menjadi data class.
- 3. Saran yang bisa dikoreksi.** Label Saran AI, bisa diubah, dan tidak tersimpan otomatis.

4. Sepuluh kasus uji. Tabel kasus dan akurasi di README, diperbarui setiap kali prompt berubah.

CONTOH Contoh dari LaporanKampus

Deskripsi menjadi saran kategori, gedung, dan urgensi di tab Asisten. Pelapor menekan Pakai, mengubah urgensi dari Tinggi ke Sedang karena tahu lantainya sudah dikeringkan, lalu mengirim laporannya sendiri.

Checkpoint minggu 9

No	Yang diselesaikan
1	Satu panggilan Gemini dengan keluaran JSON sesuai skema
2	Hasil tampil sebagai saran yang bisa diubah atau diabaikan
3	Kunci di local.properties, tidak ada di riwayat commit

Push ke repository kelompok. README memuat prompt, skema, dan tangkapan layar kartu saran AI. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Tes lisan 2 di pertemuan 13 juga meminta keluaran terstruktur yang bisa dikoreksi, terpasang di alur aplikasi kelompok.

Rubrik Laporan hasil proyek

Bahan checkpoint ini sekaligus isi **Laporan hasil proyek**, yang dikerjakan berkelompok dan dikumpulkan sebelum pertemuan 16.

Kriteria	Sangat baik, 85 sampai 100	Baik, 70 sampai 84	Cukup, 55 sampai 69	Kurang, di bawah 55
Laporan fitur AI, CPMK0503, 5%	Prompt, skema keluaran, evaluasi 10 kasus uji, dan batasan dibahas	Lengkap tanpa evaluasi terukur	Hanya deskripsi fitur	Tidak ada laporan

Bobotnya 5% dari nilai akhir, seluruhnya untuk CPMK0503. Nilai kriteria dikalikan bobotnya, dan plagiat bernilai 0. Bantuan AI boleh dipakai selama kalian paham setiap barisnya, dan siapa saja bisa diminta menjelaskan satu bagian kodenya.

PENUTUP

Rangkuman

Prompt yang bisa diandalkan punya empat bagian: **peran, konteks, format, dan contoh**, dengan masukan pengguna di akhir. Prompt disusun oleh fungsi biasa di kode, memakai daftar pilihan yang sama dengan form.

Gemini dipanggil dengan POST ke `generateContent`. **`responseMimeType`** meminta jawaban JSON, dan **`responseSchema`** menentukan nama field, tipe, dan pilihan yang boleh dipakai lewat enum. Skema ditulis sebagai data class, dan jawabannya dibaca dua kali: badan HTTP menjadi `GeminiResponse`, lalu teks di dalamnya menjadi `SaranLaporan`.

Kunci API disimpan di `local.properties`, dibaca lewat `BuildConfig` di Android atau variabel lingkungan di desktop, dan tidak pernah masuk riwayat commit.

`AsistenLaporan` adalah interface dengan versi Gemini dan versi palsu. Error menjadi `Result.failure`, dengan `CancellationException` tetap dilempar ulang.

Hasil AI tampil sebagai **saran** yang diberi label, bisa diubah, dan tidak tersimpan otomatis. Saat AI gagal, form tetap bisa diisi. Data pribadi tidak dikirim ke AI. Akurasi diukur dengan sepuluh kasus uji yang jawabannya disusun sebelum melihat hasil AI.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Sebutkan empat bagian prompt dan contohnya di LaporKampus.
2. Apa gunanya meminta keluaran JSON dengan skema?
3. Kenapa gedung juga diberi enum di skema?
4. Kenapa kunci API tidak boleh ada di repository, dan apa yang harus dilakukan kalau terlanjur di-push?
5. Kenapa runCatching kurang tepat dipakai di dalam suspend function?
6. Kenapa hasil AI ditampilkan sebagai saran yang bisa dikoreksi?
7. Kenapa jawaban benar untuk kasus uji ditulis sebelum melihat jawaban AI?

JAWABAN Kunci jawaban

1. Peran: membantu admin fasilitas kampus. Konteks: daftar gedung, kategori, dan arti urgensi. Format: hanya JSON sesuai skema. Contoh: satu deskripsi beserta jawabannya.
2. Supaya jawaban bisa langsung dibaca menjadi data class dan diperiksa, tanpa trik membuang kalimat pengantar atau tanda kode.
3. Supaya AI tidak menulis nama gedung dengan cara yang berbeda-beda, dan tidak mengarang gedung yang tidak ada.
4. Repository bisa dibaca orang lain, dan kunci bisa disalahgunakan sampai kuota habis. Kalau terlanjur di-push, hapus kunci itu di AI Studio dan buat kunci baru.
5. Karena runCatching ikut menangkap CancellationException, sehingga pembatalan coroutine tertelan.
6. Karena AI bisa salah, dan keputusan tetap harus di tangan pengguna.
7. Supaya kunci jawaban tidak tanpa sadar menyesuaikan diri dengan jawaban AI, sehingga akurasi jujur.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
ai.google.dev, Gemini API quickstart	Membuat kunci dan permintaan pertama
ai.google.dev, Structured output	responseMimeType, responseSchema, dan enum
ai.google.dev, Prompt design strategies	Menyusun prompt yang jelas
ai.google.dev, Models	Daftar nama model yang berlaku saat ini
aistudio.google.com	Mencoba prompt sebelum ditulis di kode

CONTOH **Masalah Pertemuan 10**

Setiap kali menambah fitur, ada fitur lama yang diam-diam rusak dan baru ketahuan saat demo. Bagaimana memastikan kode tetap benar tanpa mengetes manual satu per satu? Bawa fitur AI kelompok dan AsistenPalsu kalian. Minggu depan keduanya menjadi bahan unit test pertama.